# Assignment 8: Finding and Tracking Objects

This Assignment has two parts. In Part 1, we will build a ROS2 action server and client for object tracking. The action server will be able to track any of the 80 COCO dataset classes using the YOLOv8 Isaac ROS object detection pipeline we built in class. A ROS2 action client can request from the action server which object to track and receive feedback during tracking. It also gets a result on whether the tracking was successful. In Part 1, we will ensure that the software stack for the action server and client is working correctly before proceeding to Part 2, in which we will implement tracking by sending commands to the robot base to engage the motors and locate the objects.

# Part 1: "Where is the Object?"

In this part, we develop the action server and client and debug them using text outputs. We don’t need to move the robot yet until Part 2.

## Action Client Specs

- Asks for user input about the object to track (see below).
- Sends a goal to the action server with the specified object.
- Prints all feedback messages received from the action server during tracking.
- Prints the result message from the action server: "Tracking Successful!" or "Tracking Failed."
- After receiving a result message from the server, whether successful or not, the client requests new user input for an object to track and asks the server to track it.

## Action Server Specs

- Gets a goal request with a specified object to track.
- It accepts the client's request if the object belongs to one of the 80 object classes in the COCO dataset. Otherwise, it rejects it.
- After the request is accepted and the object is already in the RealSense camera's field of view (FOV), the server starts tracking it and sending feedback to the

client. The feedback should consist of short messages about the general location of the object: left, right, center, ahead, top left, bottom right, etc.
- If the object box center is anywhere around the center of the camera FOV (within ±30 pixel radius) and about 20 cm away from the robot, then the action server will complete the action and return the result message "Tracking Successful!" to the client.
- If the object's bbox center disappears from the camera FOV at any time after tracking starts, the action server will terminate the actions and return the message "Tracking Failed."
- If the requested object is not initially in the camera FOV, the action server will wait indefinitely until it appears. We will change this behavior in Part 2.
  While waiting, the action server will send the feedback message to the client: "Searching."

## Testing the Action Server Tracking

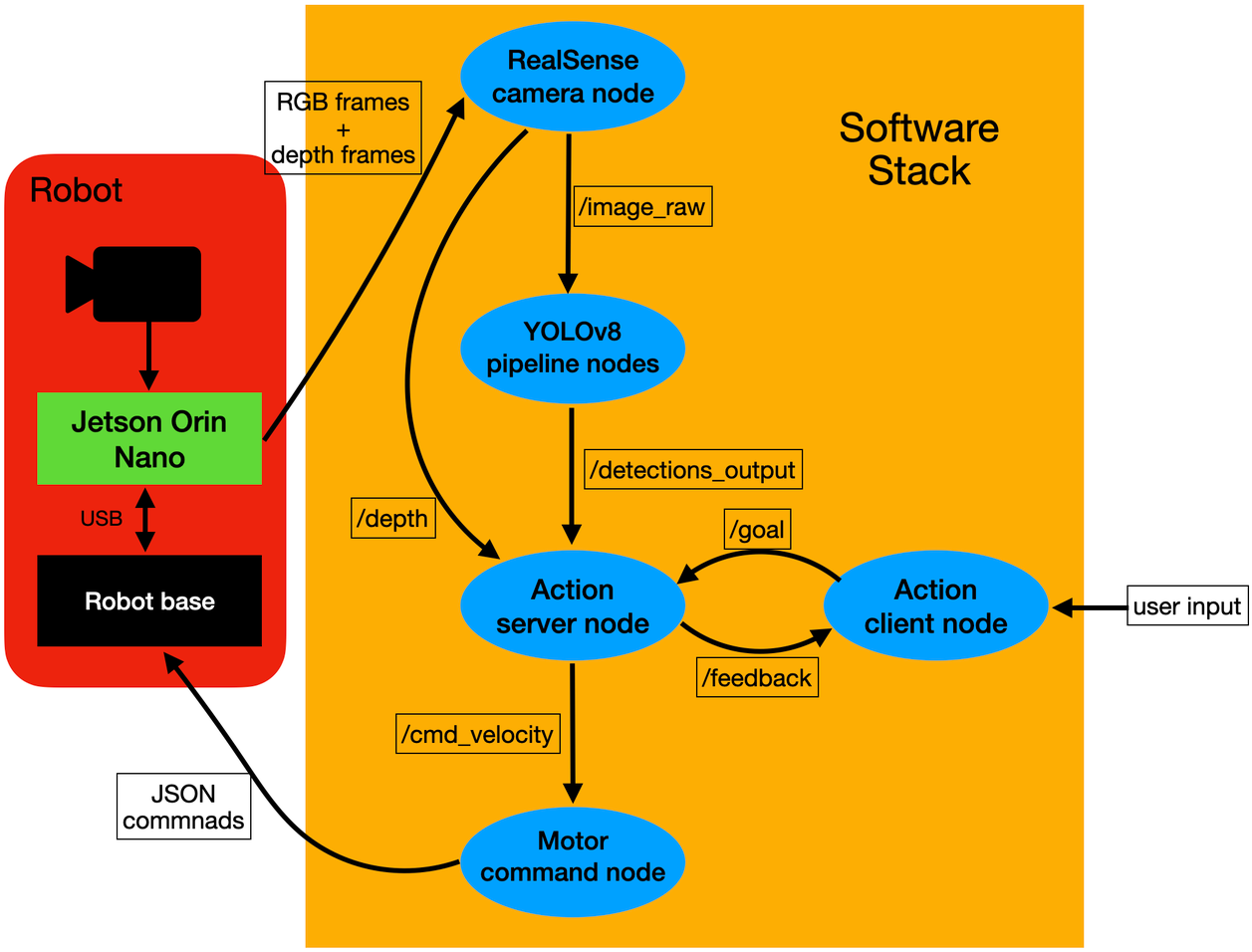
- For testing purposes, find an object from the COCO dataset—it could be a real-sized object like a toothbrush, a knife, or a cup, or it could be a miniature object like a giraffe, a zebra, a truck, or a motorcycle—and move that object manually in the FOV. Test both the client and server using text output to the screen as we did in Assignment 5.

# Part 2: "Go Find My Object"

## Tracking Algorithm

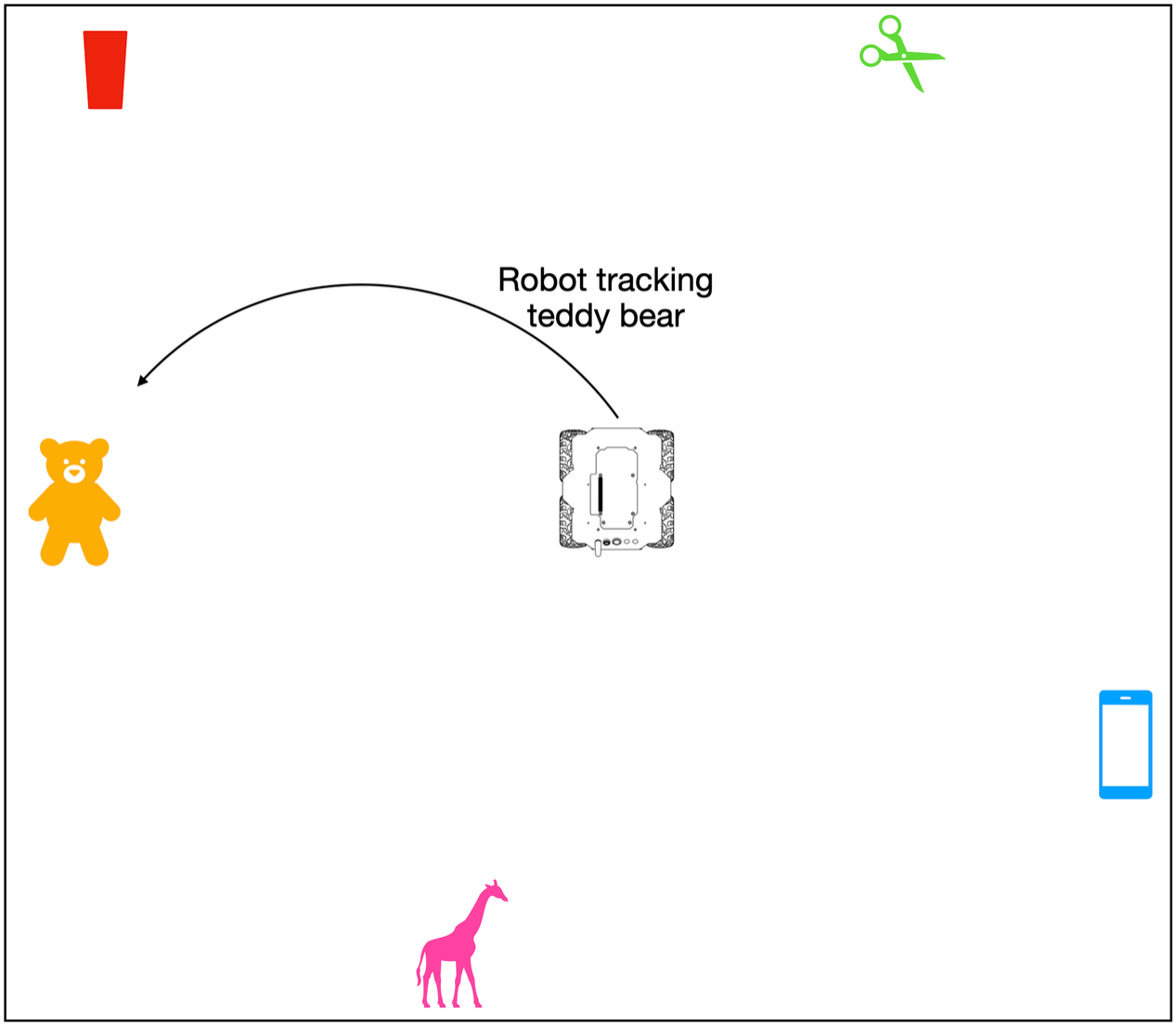
- To move the robot toward its target object, you will need to control its left and right movement. You should use the bounding box x-axis center coordinate (bbox_cx) obtained from Part 1. The bbox_cy is irrelevant here since the robot cannot move up and down in space! The right and left movement control is accomplished through the angular movement of a Twist message on the /cmd_vel topic. You can make the angular movement rates proportional to bbox_cx, positive or negative, with respect to the center of the FOV.
- Use the depth distance (dep_dist) obtained in Part 1 to control the forward linear motion. You can make the forward speed proportional to dep_dist.
- You will need to develop a state machine to decide how to move the robot toward the object selected by the Action client. For example, states can be waiting, tracking, searching (or spinning), or stopped. The state machine's inputs will be

the bbox_cx and dep_dist variables, and its outputs will be published commands to the "/cmd_vel" topic using the Twist message type. Here is the overall block diagram of ROS2 for object tracking:




*ROS2 node graph for object tracking*

An example environment scenario test object tracking of any of the COCO data set class objects:

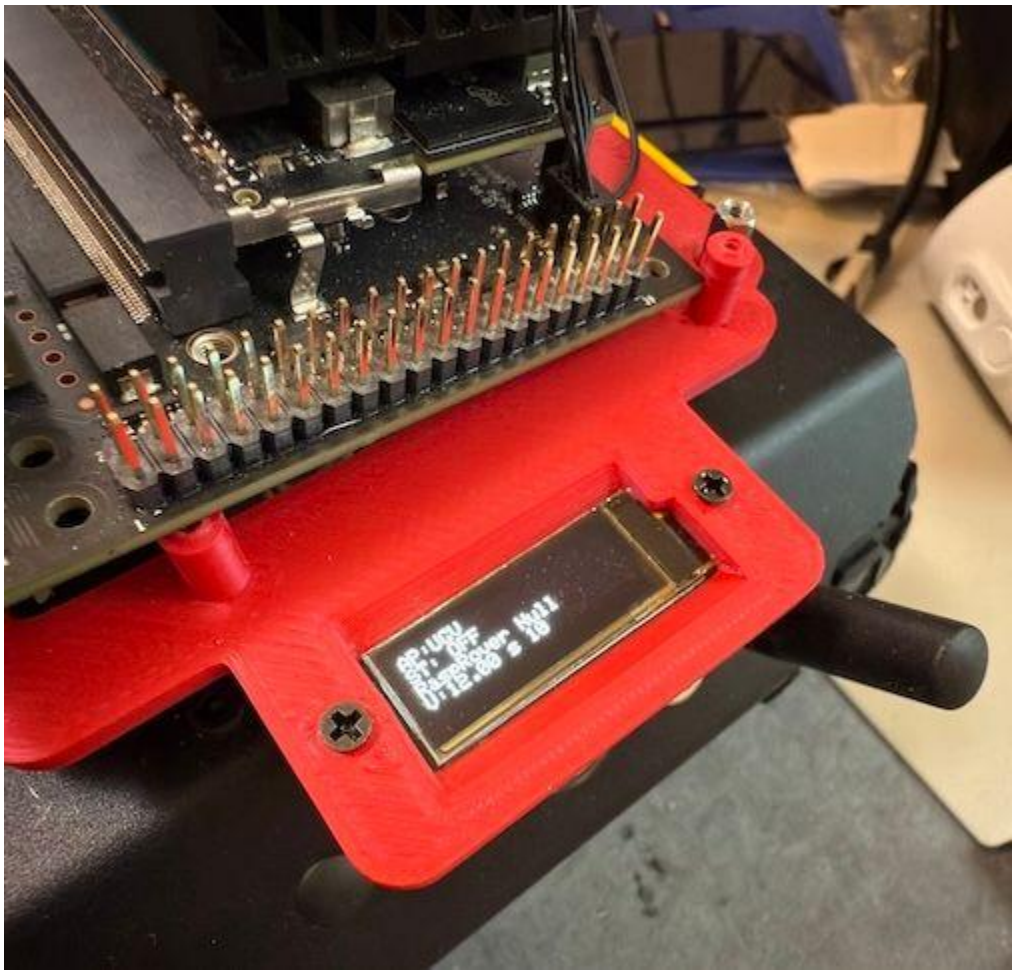



*A test environment for object tracking*

## Testing the motors on the Waveshare robot base

- First, test the motors as I outlined here.
- **Warning:** DO NOT plug the Jetson Orin Nano's power adapter into the Waveshare robot to charge the batteries. The Jetson adapter has different voltage levels that might damage the robot's base.
- To properly charge the Waveshare robot base batteries, ensure the robot USB base cable is disconnected from the Jetson. Plug the power adapter that came with the Waveshare robot into the base to charge it. When the batteries are fully charged, the power adapter LED will turn from RED to GREEN as shown below.

- Once the robot base batteries are charged, unplug the power adapter and turn on the robot base with the silver push button next to the barrel jack plug. The robot takes about 2 seconds to boot up, and the OLED screen should look like this:



- Plug the USB cable connected to the Waveshare robot into the USB-A ports on the Jetson.
- Power up the Jetson with its own power adapter (Please **DO NOT** use the Waveshare power adapter).
- Lift the robot on a stand so the wheels can run freely and do not contact the surface. This way, we can test the motor control software without worrying about the robot moving around. I have some wooden cubes cut at TechSpark that I will leave in the lab in HH A101 for this purpose.
- Start up the Jetson, and start the Docker isaac-ros container.
- We need to add the Docker admin user to the Linux “dialout” group inside the Docker container. This way, the Docker admin user has permission to access the USB port on the Jetson:

```
inside docker: $ sudo usermod -aG dialout admin
inside docker: $ exit
```

- After exiting the Docker container, restart the Jetson to make sure the group modification takes effect:
- Check that the dialout group was added and that it shows in the admin groups:

```
inside docker: $ groups
admin dialout sudo video plugdev
```

- I developed C/C++ sample test code that sends JSON commands to the Waveshare robot base, which I made available in this GitHub repo.
- Inside the Docker container, download the file test_waveshare_cmd.cpp and compile it:

```
insdie docker: $ g++ test_waveshare_cmd.cpp -o test_waveshare_cmd
```

- After making sure that the robot is lifted up on a stand and the wheels are not in contact with the surface, run the motor command test:

```
inside docker: $./test_waveshare_cmd
```

- You should see the motors turn on and the wheels spinning for about three seconds, then stop.

## Waveshare Robot Base JSON Commands

- The Waveshare robot base takes JSON commands to control the motors in the form:

```
'{"T":13,"X":0.3,"Z":2.9}'
```

  Where the T=13  is the command type for motor control, X=0.3 is the linear velocity, and Z=2.9 is the angular velocity.
- **WARNING:** The robot should not exceed a linear velocity of 0.5. It has powerful motors, and with a linear velocity above 0.5, it will be tough to control properly.
- The test_waveshare_cmd.cpp shows how to create a JSON string from the numerical values or the X and Z fields.  You can fill these fields with their respective values from the /cmd_vel topic with the Twist message interface.
- For a list of other Waveshare JSON commands, check out the Waveshare Wiki page.

## ROS2 Node to Control the Robot Motors

- Develop a ROS2 node that subscribes to the /cmd_vel topic published by your Action server. This node sends JSON commands over the USB serial connection to the robot base to move the robot.

- This node needs to be very simple and can be launched independently (with the ros2 run command) or launched from the launch file as the Action server.
- This node subscribes to the /cmd_vel topic and issues the JSON commands to the Waveshare robot base.

## Testing the Robot Object Tracking

Finally, when you think your object tracking software is thoroughly debugged, the robot is still on a stand (or jacked up), check that the wheels move in the expected directions as you wave one of the COCO class objects in front of the camera. Here is the procedure to test the physical robot movement:

1. Power off the Jetson, then disconnect it from its power supply adapter.
2. Make sure the robot base is also powered off.
3. Connect the barrel jack to the Waveshare base, as shown below, so that the Jetson can be powered up from the robot base. Then, power the robot base up by pushing the silver button on the back under the OLED screen.

3. The Jetson should also power up as well as the robot base.

**NOTE:** You don't want to debug your code for a long time while the Jetson is powered from the robot base. Doing so will drain the batteries, and you will have to wait for them to charge again before you can continue.

4. Start the YOLOv8 image pipeline, which needs to be physically connected to a computer monitor. At this point, you don't need to run the visualizer or rqt_viewer.
5. Don't start your ROS2 action, server, and motor control nodes yet. You will do that in the steps below, so continue reading.
6. Find the Jetson's IP address by either going to the WiFi settings or by using the Linux command below and picking the first IP address. Write down this IP address for the later step.

```
$ hostname -I
```

7. Disconnect the Jetson from the computer monitor. It should now be free from static cables and ready to move freely.
8. Using another computer/laptop, "ssh" to the Jetson using the IP address from above:

```
$ ssh <jetson-username>@ip_address
....
```

   Select yes if SSH prompts you to add the host to a list of known hosts, and enter your password on the Jetson.
9. From the SSH remote connection prompt, connect to the running Docker container that you started above:

```
ssh_prompt: $ docker exec -it -u admin isaac_ros_dev-aarch64-c
ontainer
```

10. From the SSH prompt, start your action server:

```
ssh_prompt (inside docker): $ ros2 launch <action_server_pkg>
<launch_file>
```

11. Create a second SSH remote connection as above, connect to the running Docker container, and start your action client.
12. Create a third SSH remote connection as above, connect to the running Docker container, and start your motor connection node.
13. Finally, remove the robot from the stand and place it on a safe surface for testing. Make sure there are no obstacles or edges that the robot can hit or fall off. In your group, position teammates around the edges of a table to prevent the robot from falling off.
14. Place an object or objects from the COCO data set within the robot view.
15. Enter a class object for the Action client to track one of the objects you placed.

16. If the robot is not performing correctly, you need to debug your code by powering off the Jetson from the SSH remote prompt.

```
ssh_prompt (inside docker): $ exit
ssh_prompt: $ sudo shutdown now
```

17. Reconnect the Jetson to the power supply adapter, power up the Jetson, and debug your code by going back to step 1.
    **NOTE:** You don't want to debug your code for a long time while the Jetson is powered by the robot base. Doing so will drain the batteries, and you will have to wait for them to charge again before you can continue.

Once your robot performs correctly, demo your work during demo day and take pride in your work! 😎

---

# Helpful information

## Launching the object detection pipeline with depth perception:

Check out the command here.

## Creating the action server and client packages:

You can create the packages the same way we did in Assignment 6, inside the Docker container under:

```
/workspaces/isaa_ros-dev/src
```

## How to change the YOLOv8 parameters:

The isaac_ros_yolov8 package is installed under /opt/ros/humshare/isaac_ros_yolov8. The launch file that controls the YOLOV8 parameters, such as objectness confidence and NMS IoU threshold, is the following path **inside the docker container**: **/opt/ros/humble/share/isaac_ros_yolov8/launch/isaac_ros_yolov8_core.launch.py**

## How to change the RealSense camera parameters:

I advise you to leave the camera resolution at 640x480 pixels at 15 FPS (or 640x480x15). This setting seems to give the best accuracy at a reasonable FPS. However, this setting can be changed in the file inside the container:

```
/opt/ros/humble/share/isaac_ros_realsense/config/realsense_mono.yaml
```

In the above file, you can specify the resolution by adding a line such as:

```
rgb_camera.profile: 640x480x15
```

The first two numbers are the width and height, and the last is the FPS.
For the depth option, you can specify parameters in the file:

```
/opt/ros/humble/share/isaac_ros_realsense/config/realsense_mono_depth.yaml
```

## RealSense camera Depth Specs:

The table below specifies the minimum Z depth for various RealSense models and resolutions. Note that the minimum Z depth for the D435i (our camera model) is a function of image resolution; at 640x480 resolution, it is 17.5 cm.

**Table 4-11. Minimum-Z Depth**

| Resolution | D410/ D415 | D430/D435/ D435i/D435f/D435if | D450/D455/ D455f/D456 | D401/D405 |
|---|---|---|---|---|
| | Min-Z (mm) | Min-Z (mm) | Min-Z (mm) | Min-Z (mm) |
| 1280x720 | 450 | 280 | 520 | 100 |
| 848X480 | 310 | 195 | 350 | 70 |
| 640x480 | 310 | 175 | 320 | - |
| 640x360 | 240 | 150 | 260 | 55 |
| 480x270 | 180 | 120 | 200 | 45 |
| 424x240 | 160 | 105 | 180 | 40 |

## How to get user input in a C++ ROS2 node:

There are at least a couple of ways:

- Spawn a thread that reads user input:

```
#include <rclcpp/rclcpp.hpp>
#include <thread>
#include <iostream>
```

```
#include <string>

class InputMonitorNode : public rclcpp::Node
{
public:
  InputMonitorNode() : Node("input_monitor_node")
  {
    RCLCPP_INFO(this->get_logger(), "Node started. Type input
below:");
    input_thread_ = std::thread(&InputMonitorNode::readInput,
this);
  }

  ~InputMonitorNode()
  {
    keep_running_ = false;
    if (input_thread_.joinable()) {
      input_thread_.join();
    }
  }

private:
  void readInput()
  {
    std::string input;
    while (rclcpp::ok() && keep_running_) {
      std::cout << ">>> ";
      std::getline(std::cin, input);
      if (!input.empty()) {
        RCLCPP_INFO(this->get_logger(), "User typed: '%s'", in
put.c_str());
```

```
        // You could publish this, call a service, etc.
      }
    }
  }

  std::thread input_thread_;
  bool keep_running_ = true;
};

int main(int argc, char *argv[])
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<InputMonitorNode>();
  rclcpp::spin(node);
  rclcpp::shutdown();
  return 0;
}
```

- Or use the rclcpp::spin_some() function call in a while loop in the “main” function:

```
int main(int argc, char *argv[])
{
  rclcpp::init(argc, argv);

  auto node = std::make_shared<SpinSomeNode>();
  rclcpp::ExecutorOptions options;
  rclcpp::executors::SingleThreadedExecutor executor(options);
  executor.add_node(node);

  // Main loop with spin_some
  while (rclcpp::ok() && other_flags) {
    executor.spin_some();  // Process any ready callbacks
```

```
    // You can do other stuff here such as read user input and set other_flags
  }

  rclcpp::shutdown();
  return 0;
}
```